

Double-Blind

A dark factory that specifies everything twice, blind.

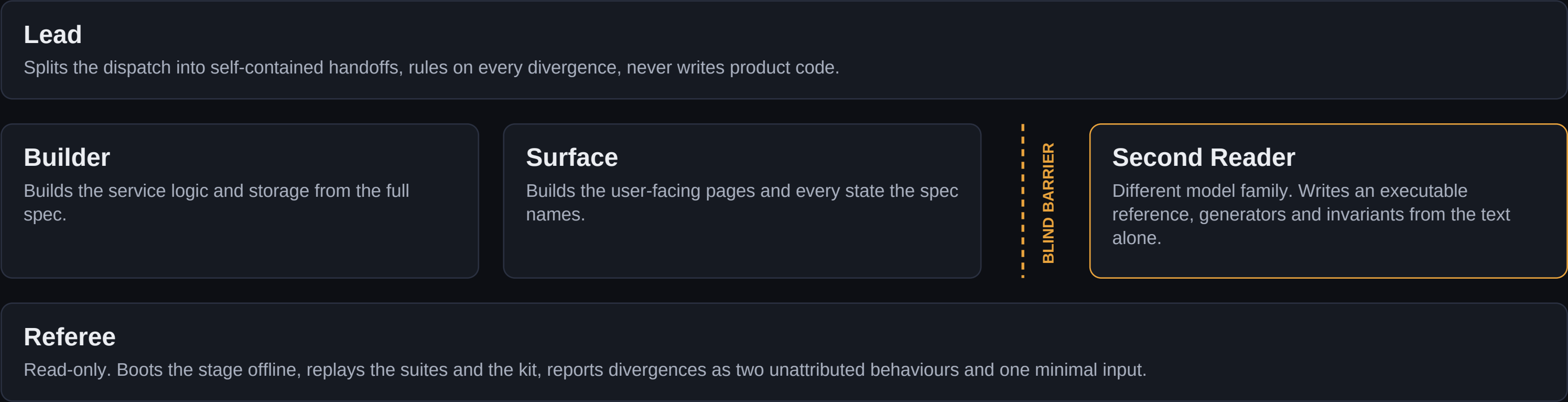
Submitted run: stage 1 not completed. One human message, 33 minutes, harness stage 0.

Abrar Ahmad · github.com/Abrar5510/dark-factory

The shared misreading

- In a planner → coder → reviewer line, the tests and the code come from one reading of the spec. A misread clause passes review every time.
- On this track the shipped checks cover only 79%, 35%, 9% and 16% of the graded suite for stages 1 to 4. "Green on the visible checks" stops meaning much almost at once.
- With no human in the run, nobody is there to resolve an ambiguous clause. The factory has to do it, and leave evidence.

Five seats, two blind readings of one spec



Mandates name no endpoint, field or error code. harness check passes the vocabulary scan.

Compare mechanically, rule by quotation

- Two blind readings. Builder and Second Reader each get the complete spec and never see each other's work. The Second Reader must seed faults into its own reference to prove its kit can fail.
- Mechanical comparison. The Referee runs earlier suites, this stage's suite, the next stage's suite (which must fail) and the kit. It never gives an opinion and never fixes code.
- Rule by quotation. The Lead rules each divergence as implementation wrong, reference wrong or requirements silent, quoting the deciding sentence. The Referee greps for the quote and rejects a paraphrase.
- Every ruled case becomes a regression check for later stages. Silent cases go into an ambiguity ledger.

One message in, 33 minutes, no working stage 1

- The only human input was the dispatch to @Lead, sent once in a fresh room with all five seats.
- The Lead pasted the full spec to the Builder and, separately, to the Second Reader, then chased both on the clock.
- The Builder wrote a TypeScript service; the Second Reader wrote a Python reference model, generators, invariants and a self-test from the spec alone.
- Three isolated harness checks, all failed at the Docker build. The Lead closed the stage at its cap and reported the build as red.
- Surface and Referee joined the room and made zero tool calls.

Stage 1 does not build. Harness: stage 0.

```
$ python -m harness run --track pocketful --stage 1 --mode isolated
src/database.ts(1,8): error TS2395: Individual declarations in merged
declaration 'Database' must be all exported or all local.
The command 'npm run build' returned a non-zero code: 2
highest contiguous stage: 0
```

```
$ python -m harness check --track pocketful
gate 2: two of your own seats must exchange messages using each
other's @handles, with a reply in each direction
```

Run against the committed submission. The seats' own three isolated checks failed the same way.

What the room and the history actually show

- Autonomy held: one human message, no steering, approvals or reruns.
- Collaboration was thin: only the Lead posted text. Builder and Second Reader never replied with an @handle, so harness check fails gate 2.
- Provenance is weak: all four commits are authored by Lead, including product code its mandate forbids.
- No divergence was found or ruled, because the service never ran.

```
$ git log --format='%an | %s'
Lead | Stage 1: Add final report with check results and deferrals
Lead | Stage 1: Add lessons and deferrals document
Lead | Stage 1: Add stub routes and fix TypeScript compilation
Lead | Stage 1: Initial implementation structure
```

MEASURED COST

33 minutes, 1 human message, 188 tool calls

Seat	Model	Tool calls
Lead	DeepSeek-V3.2 (mandate says Kimi-K2.5)	106
Builder	DeepSeek-V3.2	62
Second Reader	Kimi-K2.5	20
Surface	DeepSeek-V3.2	0
Referee	DeepSeek-V3.2	0

Tokens and USD were not measured: the Band-hosted seats wrote no usage events to room.json.
Dispatch 04:04 UTC, first commit 04:28, Lead closed the stage 04:37. 445 room events.

Honest limits

- The submitted run did not complete stage 1. The committed service does not compile and the harness scores it stage 0.
- The catch-and-recover design is untested: the Referee was never engaged and the kit was never run against a service.
- The Lead likely ran on DeepSeek, not the Kimi model its mandate names, and we could not confirm the Band-hosted seats loaded their mandates.
- The blind barrier is by convention only; nothing technically stops the Builder opening the verification repository.
- The run was cut to 45 minutes and stopped at the cap. About 14 minutes of the room were not captured live in the video.
- No completed rehearsal existed before the submitted run.

SUBMISSION

Everything a judge needs is in one public repo

- Repository: github.com/Abrar5510/dark-factory
- FACTORY.md: seat setup, design rationale, measured costs, catch and recover.
- mandates/: one generic standing instruction per seat.
- room.json: the full Band room export for the submitted run.
- stage-1/: the seats' committed work. It does not build; no later stage was attempted.
- Video: BAND Desktop room recording plus walkthrough.